

Honeytrap Infrastructure Implementation Report

SOC Threat Collection & Network Detection Platform

1. Project Overview

As part of the Hybrid SOC project, a dedicated **Honeytrap Sensor** was designed and deployed to collect attacker activity, detect malicious network behavior, and generate high-quality telemetry for centralized analysis within the SOC.

Instead of deploying a complete T-Pot distribution, a **lightweight modular architecture** was chosen to avoid redundant components because the project already contains a dedicated ELK/SIEM infrastructure. This approach reduces resource consumption while providing greater flexibility and demonstrating individual component integration.

2. Design Objectives

The honeytrap platform was designed to achieve the following objectives:

- Capture attacker behavior
 - Record brute-force attacks
 - Observe malware delivery attempts
 - Detect network attacks
 - Generate high-quality security logs
 - Integrate with centralized SIEM
 - Support future threat intelligence enrichment using MISP
 - Simulate enterprise SOC sensor architecture
-

3. Initial Design Decisions

Several deployment approaches were evaluated before implementation.

Considered Solutions

- T-Pot
- Cowrie
- Dionaea
- Heralding
- AIDE
- Suricata IDS
- Wazuh

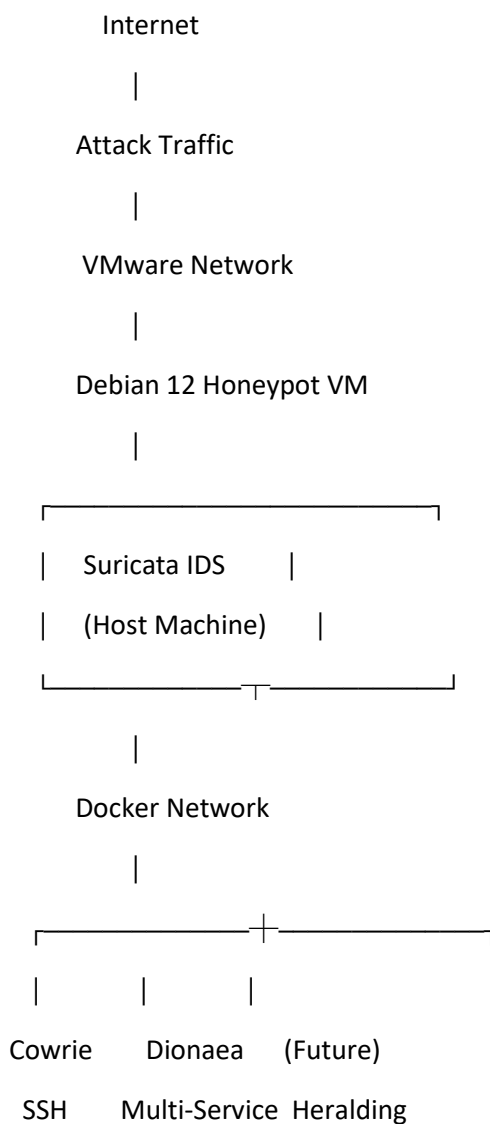
- ELK Integration

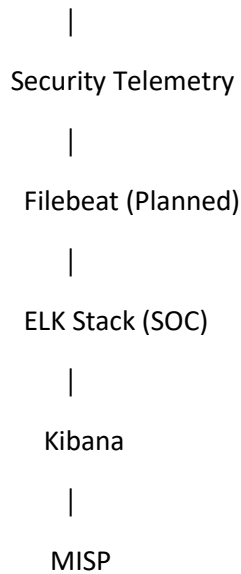
Important Design Decisions

Instead of installing the complete T-Pot platform, a custom deployment was selected because:

- Existing ELK infrastructure already provides log management.
- No need to duplicate Elasticsearch or Kibana.
- Lower RAM and CPU utilization.
- Easier troubleshooting.
- Easier maintenance.
- Better understanding of individual security components.
- More enterprise-like deployment.

4. Final Architecture





5. Base Operating System

The honeypot sensor was deployed on:

- Debian GNU/Linux 12 (Bookworm)
- VMware Virtual Platform
- 4 vCPUs
- 8 GB RAM
- Dedicated honeypot VM

The installation was performed as a minimal server installation using:

- SSH Server
- Standard System Utilities

A graphical environment was intentionally excluded to minimize resource usage.

6. Docker Platform

Docker was selected for honeypot deployment because it provides:

- Service isolation
- Easy upgrades
- Easy recovery
- Independent deployment
- Simplified maintenance

Docker Compose was installed for orchestration.

Validation included:

- Docker Engine installation
 - Docker Compose installation
 - Docker daemon verification
 - Hello World container execution
-

7. Project Directory Structure

/opt/soc-honeypot

```
├── cowrie
├── dionaea
├── heralding
├── suricata
├── aide
├── filebeat
└── configs
```

A dedicated Docker bridge network (**soc-network**) was also created to isolate honeypot containers.

8. Cowrie Honeypot Deployment

Cowrie was selected as the SSH/Telnet deception platform.

Purpose

- SSH Honeypot
- Credential harvesting
- Brute-force detection
- Fake Linux shell
- Command logging

Initially, deployment encountered issues because bind-mounted directories prevented Cowrie from creating:

- UUID files
- SSH host keys
- Log directories

The deployment strategy was revised to use **Docker named volumes**, resolving:

- Permission issues
 - Missing directory structures
 - Persistent storage problems
-

Successful Validation

Cowrie successfully:

- Accepted SSH connections
- Generated sensor UUID
- Started JSON logging
- Presented a fake Linux shell
- Logged attacker credentials
- Recorded executed commands

Testing confirmed:

- Failed logins
 - Successful fake logins
 - Command execution
 - Session logging
-

9. Cowrie Telemetry

Captured information includes:

- Source IP
- Username
- Password
- SSH Client Version
- Session ID
- Timestamp
- Executed Commands
- Session Duration

This telemetry is valuable for brute-force detection and attacker behavior analysis.

10. Suricata Deployment

Suricata was deployed directly on the host instead of inside Docker because it requires direct access to the network interface.

Advantages:

- Full packet capture
 - Better performance
 - Easier AF_PACKET configuration
 - Direct network visibility
-

11. Suricata Configuration

The primary monitoring interface was configured as:

ens33

Packet capture mode:

AF_PACKET

HOME_NET was configured for the internal VMware subnet.

12. Issues Encountered During Suricata Deployment

Package Availability

Initially, the Suricata package was unavailable due to repository configuration. After correcting the package sources, installation proceeded successfully.

Interface Error

The service initially failed because it attempted to monitor:

eth0

The correct VMware interface was:

ens33

Updating the configuration resolved the startup failure.

13. Rule Configuration

Emerging Threats Open Rules were installed.

Statistics:

- More than **52,000 IDS signatures** loaded successfully.

Configuration validation confirmed:

- Rule loading
 - YAML syntax
 - Packet capture configuration
-

14. Suricata Validation

Traffic inspection successfully identified:

- SSH traffic
- FTP traffic
- HTTP traffic

Suricata generated protocol events in:

eve.json

Validation confirmed that traffic directed to the honeypot was inspected before reaching Docker containers.

15. Dionaea Deployment

Dionaea was deployed as a multi-service honeypot.

Purpose:

- Malware capture
 - Exploit detection
 - Service emulation
-

Enabled Services

Dionaea successfully emulated:

- FTP
- HTTP
- SMB
- MSSQL
- MySQL
- SIP
- TFTP
- PPTP

- Printer
- UPnP

These services increase the attack surface and enable observation of a wider range of attacks beyond SSH.

16. Persistent Storage

Persistent Docker volumes were configured for:

- Logs
 - Databases
 - Malware samples
 - Service state
-

17. Dionaea Database

The honeypot generated a SQLite database containing structured attack telemetry.

Verified tables included:

- connections
- downloads
- offers
- logins
- resolves
- p0fs
- virustotals
- mysql_commands
- mssql_commands
- sip_commands
- mqtt_fingerprints

This demonstrates that Dionaea initialized correctly and supports protocol-specific forensic analysis.

18. Network Detection Workflow

Traffic follows the sequence:

Attacker

|
Suricata IDS

|
Docker Network

|
Cowrie / Dionaea

|
Security Logs

This layered design provides both:

- Network-level visibility (Suricata)
 - Application-level visibility (Honeypots)
-

19. Major Findings

Cowrie

Successfully captures:

- SSH brute-force attacks
 - Password attempts
 - Interactive attacker commands
 - Fake shell sessions
-

Dionaea

Successfully emulates:

- Multiple vulnerable services
 - Service banners
 - Malware delivery infrastructure
 - Attack databases
-

Suricata

Successfully detects:

- SSH protocol
- FTP protocol

- HTTP protocol
 - Network metadata
 - Packet-level events
-

20. Security Benefits

The deployed platform enables:

- Threat collection
 - Brute-force monitoring
 - Malware observation
 - Network intrusion detection
 - Protocol analysis
 - Attack telemetry collection
 - SOC investigation support
 - Security event generation
-

21. Components Completed

Component	Status
Debian 12 Base Server	✓
Docker Engine	✓
Docker Compose	✓
Docker Network	✓
Cowrie SSH Honeypot	✓
Dionaea Multi-Service Honeypot	✓
Suricata IDS	✓
AF_PACKET Capture	✓
Emerging Threats Rules	✓
Protocol Detection Validation	✓
Persistent Storage	✓

Component	Status
SQLite Telemetry	

22. Planned Enhancements

The following components were planned for subsequent phases:

- Filebeat for centralized log forwarding
 - Elasticsearch integration
 - Kibana dashboards for visualization
 - MISP threat intelligence enrichment
 - AIDE (File Integrity Monitoring)
 - Heralding multi-protocol authentication honeypot
 - Cross-correlation of honeypot and IDS events
 - Automated SOC alerting and dashboards
-

23. Overall Outcome

A modular, enterprise-style honeypot sensor platform was successfully built using **Debian 12**, **Docker**, **Cowrie**, **Dionaea**, and **Suricata IDS**. The deployment demonstrates a layered security architecture where network traffic is inspected by the IDS before reaching deception services, enabling both packet-level detection and detailed attacker interaction logging. Persistent telemetry generation, protocol inspection, and attack data collection provide a strong foundation for centralized SOC monitoring through ELK and future threat intelligence correlation with MISP. This implementation showcases practical skills in Linux administration, containerized security deployments, intrusion detection, deception technologies, network monitoring, and SOC engineering.

Yes. This implementation is too large for a single response. I'll break it into well-documented phases similar to enterprise deployment documentation.

Honeypot Deployment Documentation

Phase 1 – Base Operating System & Docker Environment

Objective

Prepare a lightweight Debian server that will host all honeypot services while minimizing unnecessary resource consumption.

Step 1 – Install Debian 12

Installed **Debian GNU/Linux 12 (Bookworm)** using the NetInstaller.

Installation Type

Selected:

✓ SSH Server

✓ Standard System Utilities

Not selected:

Desktop Environment

GNOME

KDE

Web Server

Mail Server

Reason:

- Lower RAM usage
- Better performance
- Reduced attack surface
- Optimized for server workloads

Step 2 – Verify System Information

Commands executed

hostnamectl

ip addr

free -h

lscpu | grep "CPU(s)"

Verified:

- Debian 12
- VMware Virtual Platform
- 4 vCPU
- 8 GB RAM
- ens33 network interface

- Dynamic IP allocation

Step 3 – Install sudo

Minimal Debian installation did not include sudo.

Install:

su -

apt update

apt install sudo -y

Add user to sudo group

usermod -aG sudo honey

Verify

groups honey

Expected output

sudo

Logout/Login

exit

Verify

sudo whoami

Expected

root

Step 4 – Update System

sudo apt update

sudo apt full-upgrade -y

Step 5 – Install Basic Packages

sudo apt install -y \

curl \

wget \
git \
vim \
nano \
htop \
net-tools \
tree \
zip \
unzip \
ca-certificates \
gnupg \
lsb-release \
apt-transport-https \
software-properties-common

Purpose

- Administration
- Networking
- Docker installation
- Package management

Step 6 – Verify Time Synchronization

timedatectl

Verified

System clock synchronized: yes

Accurate timestamps are required for SOC log correlation.

Step 7 – VMware Snapshot

Created snapshot

Debian12-Clean

Purpose

- Rollback point

- Safe experimentation
- Faster recovery

Step 8 – Install Docker

Remove previous packages

```
sudo apt remove docker docker-engine docker.io containerd runc -y
```

Install prerequisites

```
sudo apt install -y \
```

```
ca-certificates \
```

```
curl \
```

```
gnupg \
```

```
lsb-release
```

Create Docker key directory

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Import Docker GPG key

```
curl -fsSL https://download.docker.com/linux/debian/gpg | \
```

```
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Set permissions

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Add Docker Repository

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://download.docker.com/linux/debian \
```

```
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Update repositories

```
sudo apt update
```

Install Docker

```
sudo apt install -y \
```

```
docker-ce \
```

```
docker-ce-cli \
```

```
containerd.io \
```

```
docker-buildx-plugin \
```

```
docker-compose-plugin
```

Enable Docker

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Add user to Docker group

```
sudo usermod -aG docker honey
```

Refresh session

```
newgrp docker
```

Step 9 – Verify Docker

```
docker --version
```

```
docker compose version
```

```
docker run hello-world
```

Verified

- Docker Engine
- Docker Compose
- Docker daemon

- Internet connectivity
- Docker Hub access

Step 10 – Create Project Structure

Created project directory

```
sudo mkdir -p /opt/soc-honeypot
```

```
sudo chown -R honey:honey /opt/soc-honeypot
```

Directory structure

```
cd /opt/soc-honeypot
```

```
mkdir \  
cowrie \  
dionaea \  
heralding \  
suricata \  
aide \  
filebeat \  
configs
```

Verify

```
tree /opt/soc-honeypot
```

Expected

```
/opt/soc-honeypot
```

```
├── aide  
├── configs  
├── cowrie  
├── dionaea  
└── filebeat
```

└─ heralding

└─ suricata

Step 11 – Create Docker Network

docker network create soc-network

Verify

docker network ls

Expected

soc-network

Phase 1 Outcome

Successfully prepared the honeypot host with:

- Debian 12 Server
 - Docker Engine
 - Docker Compose
 - Minimal server configuration
 - Docker bridge network
 - Organized project structure
 - Dedicated honeypot workspace
 - VMware recovery snapshot
-

Next: Phase 2 – Cowrie SSH Honeypot Deployment, including the initial deployment, troubleshooting of volume/permission issues, migration to Docker named volumes, validation of attacker interactions, and command-by-command implementation.

Honeypot Deployment Documentation

Phase 2 – Cowrie SSH Honeypot Deployment

Objective

Deploy an SSH deception platform capable of:

- Capturing brute-force attacks
- Logging attacker credentials
- Recording executed commands
- Providing an interactive fake Linux shell

- Generating telemetry for SOC analysis

Step 1 – Create Docker Compose File

Navigate to project directory

```
cd /opt/soc-honeypot
```

```
nano docker-compose.yml
```

Initial configuration

```
services:
```

```
  cowrie:
```

```
    image: cowrie/cowrie:latest
```

```
    container_name: cowrie
```

```
    restart: unless-stopped
```

```
ports:
```

```
  - "2222:2222"
```

```
volumes:
```

```
  - ./cowrie/etc:/cowrie/cowrie-git/etc
```

```
  - ./cowrie/var:/cowrie/cowrie-git/var
```

```
networks:
```

```
  - soc-network
```

```
networks:
```

```
  soc-network:
```

```
    external: true
```

Step 2 – Pull Docker Image

```
docker compose pull
```

Step 3 – Deploy Cowrie

`docker compose up -d`

Step 4 – Verify Container

`docker ps`

Expected

cowrie

STATUS : Up

PORT : 2222

Step 5 – Check Logs

`docker logs -f cowrie`

Initial Deployment Issues

The first deployment failed.

Observed errors

Permission denied when attempting to write uuid

Permission denied

No such file or directory

`var/log/cowrie/cowrie.json`

`var/lib/cowrie/ssh_host_rsa_key.pub`

Root Cause Analysis

The deployment initially used **bind-mounted host directories**.

Problems discovered:

- Host directories were empty.
- Required directory structure inside the container was overwritten.

- Cowrie could not create:
 - UUID
 - SSH keys
 - Log files
 - Runtime directories

Additionally,

etc/

var/

were owned by

root

instead of the container user.

Troubleshooting Performed

Stop Deployment

docker compose down

Inspect Directory Structure

tree /opt/soc-honeypot/cowrie

Check Permissions

ls -ld /opt/soc-honeypot/cowrie

ls -lR /opt/soc-honeypot/cowrie

Verify Compose File

cat docker-compose.yml

Verify Image User

docker image inspect cowrie/cowrie:latest --format '{{.Config.User}}'

Output

cowrie

This confirmed the container runs as a non-root user.

Solution

The deployment strategy was changed.

Instead of bind mounts,

Docker Named Volumes were used.

Updated compose file

services:

cowrie:

image: cowrie/cowrie:latest

container_name: cowrie

restart: unless-stopped

ports:

- "2222:2222"

volumes:

- cowrie-etc:/cowrie/cowrie-git/etc

- cowrie-var:/cowrie/cowrie-git/var

networks:

- soc-network

volumes:

cowrie-etc:

cowrie-var:

networks:

soc-network:

external: true

Redeployment

Remove previous deployment

`docker compose down`

`docker rm -f cowrie`

Deploy again

`docker compose up -d`

Validate Deployment

`docker ps`

Logs

`docker logs -f cowrie`

Successful startup

Loaded output engine: jsonlog

CowrieSSHFactory starting on 2222

Ready to accept SSH connections

Test SSH Honeypot

From another machine

`ssh -p 2222 root@<HONEYPOT-IP>`

Example

`ssh -p 2222 root@192.168.74.131`

Passwords tested

root

admin

password

123456

toor

Validation

Cowrie successfully generated:

- Fake login screen
- Fake Linux shell
- Session ID
- Authentication events

Validate Docker Logs

docker logs cowrie

Observed

Ready to accept SSH connections

New connection

login attempt

session closed

Validate SSH Telemetry

Observed information

- Source IP
- SSH Version
- Username
- Password
- Session Start
- Session End

- Commands Entered
-

Architecture After Deployment

Attacker

|

SSH Connection

|

Docker

|

Cowrie

|

JSON Logs

Findings

During deployment it was discovered that:

- Newer Cowrie Docker images use a minimal container image.
 - Standard Linux utilities (ls, tail, bash) are not available inside the container.
 - Runtime state should be stored using Docker volumes rather than bind-mounted empty host directories.
 - Named volumes preserve the internal directory structure expected by the application and avoid permission issues.
-

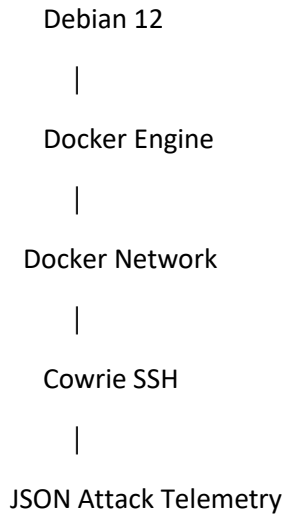
Phase 2 Outcome

Successfully deployed and validated:

- ☒ Cowrie SSH Honeypot
- ☒ Docker named volumes for persistence
- ☒ JSON logging
- ☒ Fake SSH service
- ☒ Interactive fake shell
- ☒ Credential capture
- ☒ Command logging

- ☒ Session recording
 - ☒ Persistent honeypot storage
-

Current Architecture



The next phase is **Phase 3 – Suricata IDS Deployment**, which includes installing Suricata, troubleshooting repository and interface issues, configuring AF_PACKET, loading the Emerging Threats ruleset (52K+ signatures), and validating network traffic inspection alongside the Cowrie honeypot.

Honeypot Deployment Documentation

Phase 3 – Suricata IDS Deployment & Configuration

Objective

Deploy a **Network Intrusion Detection System (NIDS)** to inspect all traffic reaching the honeypot services.

The goal was to provide:

- Network-level visibility
 - Protocol detection
 - IDS alert generation
 - Packet inspection
 - Security event logging
 - Integration with the honeypot platform
-

Design Decision

Initially, two deployment options were considered:

Option 1

Deploy Suricata inside Docker.

Option 2 (Selected)

Deploy Suricata directly on the Debian host.

Reason for Selection

Running Suricata on the host provides:

- Direct NIC access
- Better packet capture performance
- Simplified AF_PACKET configuration
- Complete visibility into all Docker container traffic
- Easier integration with Filebeat

Final architecture:

Internet

|

ens33 NIC

|

Suricata IDS

|

Docker Network

|

├─ Cowrie

├─ Dionaea

└─ Heralding (Future)

Step 1 – Install Suricata

Initially attempted:

`sudo apt update`

`sudo apt install suricata -y`

Issue Encountered

Installation failed.

Error:

Package suricata is not available

Package 'suricata' has no installation candidate

Troubleshooting

Verified Debian repositories.

Commands executed:

```
cat /etc/apt/sources.list
```

```
ls -l /etc/apt/sources.list.d/
```

```
grep -R "^deb" /etc/apt/sources.list /etc/apt/sources.list.d/
```

```
apt policy suricata
```

Observed:

Candidate: (none)

Repository Refresh

APT cache was refreshed.

```
sudo rm -rf /var/lib/apt/lists/*
```

```
sudo apt clean
```

```
sudo apt update
```

After correcting the repository state, the package became available and installation completed successfully.

Step 2 – Verify Installation

```
suricata --build-info
```

Verified:

- Installed version
 - Build options
 - Detection engine
-

Step 3 – Verify Service

```
sudo systemctl status suricata
```

Issue Encountered

The service failed.

Output:

Active: failed

Diagnostic Commands

Configuration validation:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Service logs:

```
sudo journalctl -u suricata -n 50 --no-pager
```

Runtime logs:

```
sudo tail -50 /var/log/suricata/suricata.log
```

Root Cause

Suricata attempted to monitor:

eth0

However, the VMware network interface was:

ens33

Step 4 – Configure AF_PACKET

Open configuration

```
sudo nano /etc/suricata/suricata.yaml
```

Locate:

af-packet:

Original configuration:

af-packet:

- interface: eth0

Updated configuration:

af-packet:

- interface: ens33

Verify Configuration

```
grep -n "interface:" /etc/suricata/suricata.yaml
```

Confirmed:

```
interface: ens33
```

Validate Configuration

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Successful output:

Configuration provided was successfully loaded.

Step 5 – Restart Service

```
sudo systemctl restart suricata
```

Verify:

```
sudo systemctl status suricata --no-pager
```

Confirmed:

Active: active (running)

Step 6 – Install Detection Rules

Initially, Suricata reported:

No rule files match

suricata.rules

Meaning:

- IDS engine running
 - No signatures loaded
-

Verify Rule Sources

```
sudo suricata-update list-sources
```

Available sources included:

- Emerging Threats Open
 - Abuse.ch
 - URLHaus
 - Secureworks
 - OISF
 - PAW Rules
 - Stamus Networks
-

Download Rules

```
sudo suricata-update
```

Restart

```
sudo systemctl restart suricata
```

Verify Rules

```
ls -lh /var/lib/suricata/rules/
```

Observed:

classification.config

suricata.rules

Count IDS Signatures

```
grep -c "^alert" /var/lib/suricata/rules/suricata.rules
```

Result:

52094

Over **52,000 IDS signatures** were successfully installed.

Step 7 – Final Validation

Configuration test:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Output:

Configuration provided was successfully loaded.

Step 8 – Verify SSH Detection

Generate traffic.

Example:

```
ssh -p 2222 root@192.168.74.131
```

Inspect events:

```
sudo grep '"event_type":"ssh"' \  
/var/log/suricata/eve.json | tail
```

Observed:

- SSH protocol
 - Source IP
 - Destination IP
 - SSH version
 - Ports
-

Step 9 – Validate Other Protocols

FTP

```
sudo grep '"event_type":"ftp"' \  
/var/log/suricata/eve.json | tail
```

HTTP

```
sudo grep '"event_type":"http"' \  
/var/log/suricata/eve.json | tail
```

SMB


```
sudo grep '"event_type":"smb"' \  
/var/log/suricata/eve.json | tail
```

Generated Security Logs

Suricata successfully generated:

- eve.json
 - fast.log
 - stats.log
-

Traffic Flow

Attacker

|

Network Interface (ens33)

|

Suricata IDS

|

Docker Bridge

|

Cowrie

Every packet is inspected before reaching the honeypot.

Findings

Important Discovery

Dockerized honeypots do **not** reduce network visibility.

Because Suricata monitors the physical interface using AF_PACKET, all traffic destined for Docker containers is still inspected before Docker forwards it.

This enables simultaneous:

- Network detection
 - Protocol analysis
 - Honeypot interaction logging
-

Benefits Achieved

Suricata now provides:

- Network intrusion detection
 - Protocol identification
 - SSH detection
 - FTP detection
 - HTTP detection
 - Packet metadata
 - JSON event logging
 - Signature-based detection
 - IDS telemetry
-

Phase 3 Outcome

Successfully implemented:

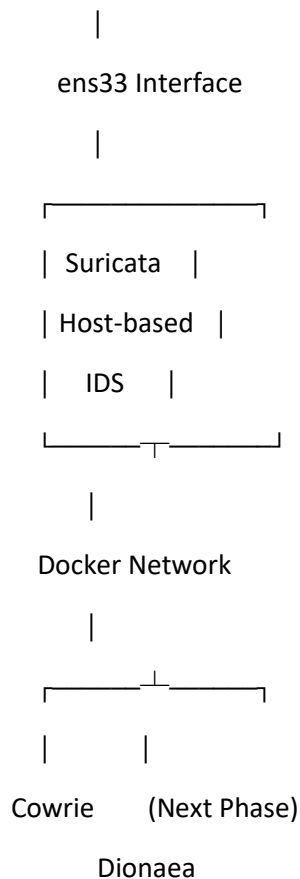
-  Suricata 7.x
 -  AF_PACKET packet capture
 -  ens33 monitoring
 -  Service startup
 -  Emerging Threats Open rules
 -  52,094 IDS signatures
 -  eve.json generation
 -  SSH protocol detection
 -  FTP protocol detection
 -  HTTP protocol detection
 -  Layered monitoring architecture
-

Updated Architecture

Internet

|

VMware Network



Next: Phase 4 – Dionaea Multi-Service Honeypot Deployment, covering Docker deployment, multi-protocol service emulation (FTP, HTTP, SMB, MSSQL, MySQL, SIP, TFTP, etc.), persistent storage configuration, SQLite-based telemetry, service validation, and integration with the existing Suricata monitoring architecture.

Honeypot Deployment Documentation

Phase 4 – Dionaea Multi-Service Honeypot Deployment

Objective

Deploy a **multi-service honeypot** capable of emulating commonly attacked network services to:

- Capture exploitation attempts
- Observe attacker behavior
- Detect malware delivery
- Record protocol-specific attacks
- Complement the existing Cowrie SSH honeypot

Unlike Cowrie, which focuses on SSH, Dionaea provides a broader attack surface by emulating multiple vulnerable services.

Design Decision

The project adopted a **modular honeypot architecture**:

- **Cowrie** → SSH interactions
- **Dionaea** → Multi-service exploitation
- **Suricata** → Network visibility
- **Heralding (planned)** → Credential harvesting

This approach increases telemetry diversity while keeping components independent.

Step 1 – Create Dionaea Workspace

Navigate to the project directory:

```
cd /opt/soc-honeypot
```

```
mkdir -p dionaea
```

```
cd dionaea
```

Step 2 – Create Persistent Docker Volumes

Persistent storage was configured to retain logs and attack data across container restarts.

```
docker volume create dionaea-logs
```

```
docker volume create dionaea-data
```

Verify:

```
docker volume ls
```

Step 3 – Create Docker Compose File

```
nano compose.yml
```

Configuration:

```
services:
```

```
  dionaea:
```

```
    image: dinotools/dionaea:latest
```

```
    container_name: dionaea
```

restart: unless-stopped

ports:

- "21:21"
- "42:42"
- "69:69/udp"
- "80:80"
- "135:135"
- "443:443"
- "445:445"
- "1433:1433"
- "1723:1723"
- "3306:3306"
- "5060:5060"
- "5060:5060/udp"

volumes:

- dionaea-logs:/opt/dionaea/var/log
- dionaea-data:/opt/dionaea/var/lib

networks:

- soc-network

volumes:

dionaea-logs:

dionaea-data:

networks:

soc-network:

external: true

Step 4 – Download Docker Image

`docker compose pull`

Step 5 – Deploy Container

`docker compose up -d`

Step 6 – Verify Deployment

Check running containers:

`docker ps`

Expected:

cowrie

dionaea

Check compose status:

`docker compose ps`

Both containers should report:

STATUS : Up

Step 7 – Verify Listening Services

Executed:

`sudo ss -tulpn | grep -E ':21|:80|:445|:135|:1433|:3306|:5060'`

Observed:

- FTP
- HTTP
- SMB
- MSSQL
- MySQL
- SIP

All ports were successfully exposed through Docker (docker-proxy).

Step 8 – Review Container Logs

```
docker logs dionaea --tail 50
```

Initialization confirmed creation of:

services-enabled

services-available

var/lib/dionaea

var/log/dionaea

Final startup message:

Starting dionaea ...

No runtime errors were observed.

Step 9 – Functional Testing

Testing was performed from another Linux VM (VM-MISP-01).

FTP

```
telnet 192.168.74.131 21
```

Observed banner:

220 DiskStation FTP server ready.

Finding: Dionaea intentionally emulates realistic service banners to deceive attackers.

HTTP

```
curl http://192.168.74.131
```

Received an HTTP response with a directory listing, confirming the HTTP service was active.

SMB

```
nc 192.168.74.131 445
```

Connection established, confirming the SMB emulation service was reachable.

Step 10 – Validate Docker Port Bindings

Executed:

`docker ps`

Confirmed published ports:

- 21
 - 42
 - 69/UDP
 - 80
 - 135
 - 443
 - 445
 - 1433
 - 1723
 - 3306
 - 5060/TCP
 - 5060/UDP
-

Step 11 – Verify Log Storage

Inside the container:

```
docker exec dionaea find /opt/dionaea/var/log -type f
```

Generated log files:

dionaea.log

dionaea-errors.log

Step 12 – Verify Persistent Database

Executed:

```
docker exec dionaea find /opt/dionaea/var -maxdepth 3 -type f
```

Observed:

dionaea.sqlite

This confirmed persistent attack telemetry storage.

Step 13 – Extract SQLite Database

Since the container did not include the sqlite3 utility:

```
docker cp dionaea:/opt/dionaea/var/lib/dionaea/dionaea.sqlite /tmp/dionaea.sqlite
```

Installed SQLite on the host:

```
sudo apt update
```

```
sudo apt install sqlite3 -y
```

Opened the database:

```
sqlite3 /tmp/dionaea.sqlite
```

Step 14 – Inspect Database

List tables:

```
.tables
```

Verified telemetry tables including:

- connections
- downloads
- offers
- logins
- resolves
- p0fs
- virustotals
- virustotalscans
- mysql_commands
- mssql_commands
- sip_commands
- mqtt_fingerprints
- dcerpcservices
- emu_profiles
- emu_services

This confirmed that Dionaea had initialized all protocol-specific storage components successfully.

Step 15 – Validate Suricata Monitoring

After interacting with Dionaea, Suricata was used to verify protocol detection.

FTP:

```
sudo grep '"event_type":"ftp"' /var/log/suricata/eve.json | tail
```

HTTP:

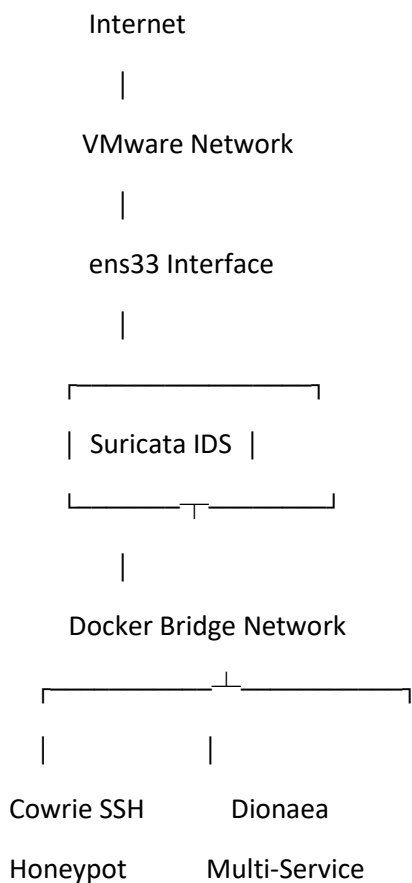
```
sudo grep '"event_type":"http"' /var/log/suricata/eve.json | tail
```

SMB:

```
sudo grep '"event_type":"smb"' /var/log/suricata/eve.json | tail
```

These checks confirmed that traffic destined for Dionaea was visible to the host-based Suricata sensor.

Architecture After Phase 4



Findings

Service Emulation

Dionaea successfully emulated multiple enterprise services, making the host appear more attractive to attackers and increasing the variety of observable attack techniques.

Persistent Telemetry

Instead of storing only flat log files, Dionaea maintains a structured SQLite database containing:

- Connection metadata
- Login attempts
- Malware download information
- Protocol-specific interactions
- VirusTotal-related tables for future enrichment

Layered Detection

The deployment demonstrated a layered monitoring model:

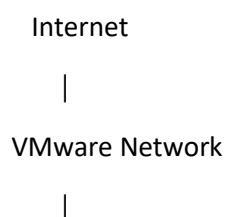
1. **Suricata** detects and logs network traffic.
2. **Dionaea** records application-layer interactions.
3. Both data sources are suitable for future ingestion into ELK.

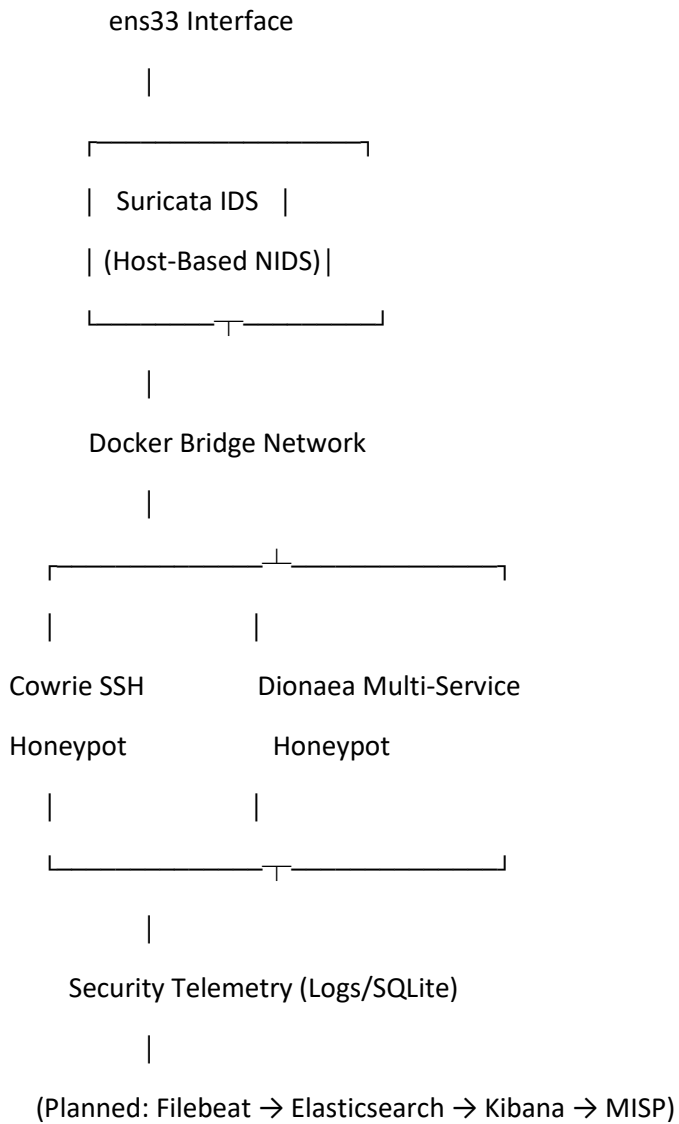
Phase 4 Outcome

Successfully implemented:

- ☒ Dionaea Multi-Service Honeypot
- ☒ Docker-based deployment
- ☒ Persistent Docker volumes
- ☒ Multi-protocol service emulation
- ☒ FTP, HTTP, SMB, MSSQL, MySQL, SIP, TFTP, PPTP, Printer, and UPnP services
- ☒ SQLite-based telemetry storage
- ☒ Runtime log generation
- ☒ Successful connectivity validation
- ☒ Integration with existing Suricata monitoring
- ☒ Modular honeypot architecture alongside Cowrie

Overall Architecture After Phase 4





This phase completed the **sensor deployment stage** of the honeypot platform. At this point, the environment consisted of two operational honeypots (Cowrie and Dionaea) monitored by a host-based Suricata IDS, forming a solid foundation for the next implementation phase: centralized log collection and SIEM integration using **Filebeat and the ELK Stack**.